Universidade da Beira Interior
Departamento de Informática

Nº 50391 - 2026 *Testing System for Java Payara APIs: An Automated Regression and CI/CD Framework for the GNSS EPOS API*

Author:

Eduardo Pires Valentim de Almeida Abrantes

Supervisor and Co-Supervisor:

Professor Paul Andrew Crocker
Vasco Senra

July 2, 2026

Acknowledgments

The completion of this project, and indeed of a large part of my academic journey, would not have been possible without the help, guidance and patience of many people. I would first like to thank my supervisor, Professor Paul Andrew Crocker, and my co-supervisor, Vasco Senra, for proposing this project and for the technical direction, the critical reviews and the trust placed in me throughout the development of the work described in this report.

A special word of gratitude goes to the staff of the SEGAL Laboratory at the Universidade da Beira Interior, Luís Carvalho and Fernando Geraldes, for the patient support with the virtual machines, the internal network and the VPN access that made the laboratory's infrastructure reachable from outside the lab and that, in practice, made the work described in this report possible.

I would also like to thank the colleagues and researchers I had the opportunity to work with at the laboratory, for the many discussions about the legacy code base, the refactoring strategy and the painful reality of regression testing when no two responses are quite alike. Those conversations shaped many of the choices presented in the following chapters.

Finally, to my family, for the constant and unconditional support that allowed me to reach this point, and to the friends and colleagues who were patient enough to listen to long monologues about differential testing, JAX-RS endpoints and YAML pipelines: thank you.

Contents

List of Figures

List of Code Listings

Acronyms

API	Application Programming Interface
CDI	*Contexts and Dependency Injection*
CI	Continuous Integration
CD	Continuous Delivery
CSV	Comma-Separated Values
EPOS	European Plate Observing System
GeoJSON	Geographic JSON
GNSS	Global Navigation Satellite System
HTTP	Hypertext Transfer Protocol
ID	Identifier
JAR	Java Archive
JAX-RS	Jakarta RESTful Web Services
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JSON-LD	JSON Linked Data
JSON-LD	JSON Linked Data
JWT	JSON Web Token
OpenAPI	OpenAPI Specification
POJO	Plain Old Java Object
REST	Representational State Transfer
RINEX	Receiver Independent Exchange Format
SEGAL	Space and Earth Geodetic Analysis Laboratory
SQL	Structured Query Language
UBI	Universidade da Beira Interior
UI	User Interface
URL	Uniform Resource Locator

VM Virtual Machine

VPN Virtual Private Network

WAR Web Application Archive

XML eXtensible Markup Language

YAML *YAML Ain't Markup Language*

Chapter

1

Introduction

1.1 Framing

This document describes the work developed in the context of the final-year project of the Computer Science and Engineering programme of the Universidade da Beira Interior (UBI). The project was proposed by, and carried out at, the Space and Earth Geodetic Analysis Laboratory (SEGAL) laboratory of UBI, under the supervision of Professor Paul Andrew Crocker and the co-supervision of Vasco Senra, with day-to-day infrastructure support from Luís Carvalho and Fernando Geraldes (responsible for the laboratory's virtual machines, internal network and Virtual Private Network (VPN) access).

SEGAL is the research laboratory of UBI that operates and maintains a Global Navigation Satellite System (GNSS) data and metadata service contributing to the European Plate Observing System (EPOS), a pan-European research infrastructure focused on solid Earth science [1, 2]. As part of the EPOS Thematic Core Service for GNSS data and products, SEGAL runs a Jakarta EE Representational State Transfer (REST) Application Programming Interface (API) that exposes GNSS station metadata and observation files (in the Receiver Independent Exchange Format (RINEX) format [3]) to scientists, operators and downstream services across Europe.

The API in question has been in operation, in various forms, for several years. Over that period the code base accumulated technical debt: enourmous methods, and files of huge scale, duplicated logic copy-pasted across resources, and a long tail of patches added by successive developers and contributors to handle individual edge cases. A complete refactor of the API was therefore initiated, in which large parts of the application were rewritten from scratch on top of Jakarta EE 11 and the Payara Platform [4, 5], while a small

core of well tested logic was carried over from the previous version.

A refactor of this magnitude poses a very specific risk: any seemingly minor change in a query, a response field or a validation rule can produce subtle, hard to detect regressions that only manifest when a particular client, a particular station or a particular file is requested. Until this project, validation of those changes was carried out manually, a developer would deploy the new version, run a few requests by hand, and compare the resulting payloads against the previous one. This is what we are trying to solve.

1.2 Motivation

The motivation for this project is the gap between the pace at which the new API is being developed and the rate at which a human can reasonably validate that each new version behaves exactly like the previous one on the cases that already worked. Manual testing of a REST API that returns thousands of records, in several formats (JavaScript Object Notation (JSON), eXtensible Markup Language (XML), Comma-Separated Values (CSV)) and through dozens of query combinations, is both error-prone and slow. This does not scale to the rhythm of a continuous refactor where commits land several times a day.

The laboratory wanted an *automated* mechanism to compare every new build of the API against the previous one and flag behavioral divergences early, before they reach the production deployment. On the other hand, the laboratory also wanted to bring this mechanism inside a real Continuous Integration (CI)/Continuous Delivery (CD) pipeline running on infrastructure that the laboratory itself controls: a free GitLab instance, paired with one or more GitLab Runners hosted inside the laboratory's network.

The choice of hosting the GitLab Runners was a necessity since the free tier only allows 400 compute minutes, and due to the nature of app, and it's boot times, a full test suit usually takes up to 5 minutes per test, and being self-hosted allows us to connect to the real database infrastructure.

Beyond the immediate value delivered to the EPOS API, designing and integrating this framework was a significant personal milestone. It challenged me to collaborate with a team on a live, rolling codebase—an experience that closely mirrors real-world software engineering. Through this process, I gained practical, hands-on experience with modern Jakarta EE features, CI/CD pipelines, self-hosted infrastructure, and regression testing against a moving baseline, equipping me with industry-relevant skills that extend far beyond the scope of this project.

1.3 Objectives

In line with the project proposal, the work pursues four objectives: to design and implement an automated regression testing framework for the Jakarta EE REST API on the Payara Platform, with multi-format response comparison (JSON, XML, CSV); to integrate it into a self-managed GitLab CI/CD pipeline running on VPN-accessible runners; use of two prototype applications not only to validate the approach before applying it to the production API, but also to explore and learn about features that could be useful for the final implementation; and to produce structured per-commit comparisons at the level of Hypertext Transfer Protocol (HTTP) responses, in a form consumable by both human developers and the CI system.

The main contributions are a reusable three-stage GitLab pipeline template for Jakarta EE applications, an HTTP-level differential test harness (`HttpDiff.java`) that boots one Payara Micro per available Web Application Archive (WAR) and performs a three-way comparison between `HEAD`, the immediate parent commit `HEAD~1` and a fixed reference commit identified by the `DIFF_FIXED_BASELINE` CI/CD variable, and two prototype Jakarta EE 11 applications retained as didactic examples for future students.

1.4 Organization of the Document

The remaining chapters of this document are organized as follows. **Chapter 2** reviews the state of the art in test frameworks, CI/CD systems, and differential testing. **Chapter 3** describes the concrete technology stack adopted during the project. **Chapter 4** explains the iterative methodology (two prototypes followed by the production application) and the overall architecture of the testing framework. **Chapter 5** applies the framework to the production API (`EPOS V4`) and walks through the pipeline, the differential harness and the reports it produces. **Chapter 6** summarizes the results, reflects on what was learned and proposes directions for future work.

1.5 Repository Access

The production API (`GNSs V4`) is hosted in a private repository restricted to laboratory members [6]. The two prototype applications developed during this work are publicly available in the author's personal namespace [7, 8].

Each repository contains a `README.md` with build instructions and (where applicable) the `.gitlab-ci.yml` configuration files discussed in Chapters 4 and 5.

Chapter

2

State of the Art

2.1 Introduction

This chapter establishes the technical background required to automatically validate a Jakarta EE REST API, build after build, against a moving reference. It explores the main families of Java testing frameworks—from standard unit tests to in-container and HTTP-level approaches—and the CI/CD orchestration tools that automate them. Because the SEGAL laboratory utilizes GitLab, special focus is given to its pipeline architecture, which directly constrained the design of this project. Finally, this chapter reviews the differential and regression testing concepts that served as the foundation for the test harness developed in Chapter 5.

2.2 Test Frameworks for the Java Ecosystem

The Java testing landscape spans multiple layers, which modern projects typically combine to ensure comprehensive coverage. At the foundational level is unit testing, where **JUnit 5** [9] serves as the de facto standard. We selected it for this project due to its first-class Maven support and its ability to generate native JUnit XML reports, which integrate seamlessly with GitLab. While **TestNG** [10] remains a viable alternative, it was deemed a legacy option for this context.

Moving up to the integration layer, Maven conventionally handles tests during its `verify` phase by executing `*IT.java` classes. To test within a container context, we evaluated **Arquillian** [11] during the first prototype. Its *micro-deployment* pattern successfully validated our initial CI/CD pipeline setup. However, Arquillian was ultimately rejected for the production API:

the application is simply too large for efficient per-test redeployments, and our methodology requires side-by-side execution rather than isolated deployments.

2.2.1 Differential Testing

The most relevant testing paradigm for this project is *differential testing*, which compares a candidate's response directly against a reference implementation rather than relying on statically defined expected outputs. A notable industry example of this concept is **Diffy** [12], which dynamically routes requests to multiple service versions to surface discrepancies side-by-side. This comparative approach addresses the exact challenge faced by the SEGAL laboratory during the refactoring of its API.

We also considered *snapshot testing*, a technique popularized by tools like Jest. However, our test infrastructure utilizes a fixed database populated with large-scale synthetic data. Freezing such massive API responses into static fixture files is highly impractical and would quickly bloat the repository. Therefore, we adopted a dynamic differential approach: rather than relying on static files or a proxy service, our harness uses the parent commit of the build under test (alongside an optional long-term reference) as the baseline. This strategy ensures comparisons remain accurate across long-running refactor branches while completely eliminating the burden of manual fixture maintenance.

2.3 Continuous Integration and Continuous Delivery

2.3.1 Foundations

CI/CD is the practice of frequently integrating code into a shared repository while keeping the codebase in a consistently deployable state through full automation. A standard deployment pipeline is structured into sequential stages typically build, test, and deploy with strict gating. A stage only executes if the previous one succeeds, ensuring that any failure immediately halts the process and alerts the development team.

The pipeline serves as the single source of truth for software quality. To prevent configuration drift between environments, a mature setup dictates that the exact artifact generated during the build stage must flow unchanged through all subsequent testing and deployment phases. This project strictly adheres to this principle: the WAR file is compiled only once during the initial

pipeline stage and is passed directly to all subsequent testing phases without modification.

2.3.2 Source Code Management with Git

Before evaluating the orchestration platforms that drive continuous integration, it is necessary to establish the underlying technology that manages the source code itself: **Git** [13]. Git is a distributed version control system where every developer maintains a complete, local copy of the repository's history. It tracks changes as a series of immutable snapshots (commits), allowing developers to work on isolated features through branching without disrupting the main codebase.

In the context of this project, Git's graph-based history is a structural requirement rather than just a collaboration tool. Because each commit strictly references its predecessor, the differential testing harness described in Section 2.2.1 can reliably identify and dynamically pull the *parent commit* to serve as the baseline for its side-by-side comparisons.

While Git provides the core version control mechanics, it is fundamentally a decentralized, command-line tool. To facilitate team collaboration at scale, organizations rely on centralized Source Code Management (SCM) platforms to host their Git repositories. Platforms such as GitHub and GitLab wrap the underlying Git protocol with web interfaces, access controls, code review mechanics (Merge Requests), and crucially for this work integrated CI/CD engines.

2.3.3 Platforms

Three major platforms dominate the modern CI/CD landscape. Evaluating them clarifies the architectural choices made for this project.

Jenkins [14] is a highly flexible, self-hosted automation server. While I have significant personal experience deploying and managing my own Jenkins infrastructure and its flexibility might have been ideal for our custom testing requirements it was ultimately rejected. Introducing a standalone CI server would impose an unnecessary maintenance burden on the SEGAL laboratory, which strongly prefers to standardize on Docker-based runners integrated with their existing tools.

GitHub Actions [15] is a popular, *YAML Ain't Markup Language* (YAML)-based CI/CD service. Although it fully supports self-hosted runners operating securely inside private networks, it was bypassed for a straightforward operational reason: the laboratory's source code management is already established on GitLab.

GitLab CI/CD [16] provides pipeline automation seamlessly integrated into the GitLab ecosystem. Pipelines are configured via a single `.gitlab-ci.yml` file, and jobs are executed by separate runner processes [17]. The SEGAL laboratory already operates a self-managed GitLab instance [18], along with a dedicated, Docker-based internal runner for the EPOS API. Because this runner is already inside the laboratory's network and can directly reach the production GNSS database without exposing it to the internet, adopting GitLab CI/CD was not an abstract preference, but the most practical choice to avoid operational friction.

2.4 Conclusion

The state of the practice in Java API testing provides a rich catalogue of tools across all layers of the stack, alongside mature CI/CD platforms to orchestrate them. From this catalogue, the project adopts JUnit 5 and the native `java.net.http.HttpClient` to build the custom test harness. A pure Java implementation was deliberately chosen over external testing tools because the existing codebase is already written in Java, ensuring the development team is working within a familiar ecosystem. Furthermore, this approach allows the harness to be executed via standard, well-documented Maven workflows, integrating seamlessly into the existing build lifecycle.

For orchestration, GitLab CI/CD was selected, a choice directly mandated by the SEGAL laboratory's established infrastructure. Conceptually, the bespoke harness described in Chapter 5 draws heavily on the differential testing principles embodied by tools like Diffy. Having established this theoretical and tooling foundation, the next chapter details the concrete technological stack upon which the implementation rests.

Chapter

3

Technologies and Tools Used

3.1 Introduction

This chapter details the specific technologies and frameworks utilized to build the solutions presented in this document. The selected stack is the result of pragmatic engineering, balancing the established architecture of the production API with modern technical requirements. Rather than enumerating these tools in isolation, the chapter examines the architecture layer by layer. It begins with API contracts and serialization formats, transitions through the Java and Jakarta EE application ecosystem and its relational databases, and finally covers the underlying servers and network tooling that support the deployment.

3.1.1 Explicit Data Serialization

Unlike typical Jakarta REST applications that rely on automatic content negotiation via `Accept` headers and transparent message body writers, the EPOS API handles serialization explicitly. Because endpoint inputs consist solely of path and query parameters rather than incoming payload bodies, the application bypasses automatic input deserialization entirely.

For outputs, the formatting logic is driven strictly by the Uniform Resource Locator (URL) path parameter. A dedicated parser component evaluates this parameter and routes the data to the appropriate conversion method. Whether the requested format is standard JSON, its JSON Linked Data (JSON-LD) or Geographic JSON (GeoJSON) variants, XML, or CSV, the parser explicitly converts the domain objects into a formatted text string. The application then manually assigns the corresponding HTTP `MediaType`

before constructing the final response.

This explicit architectural choice simplifies the validation strategy for the differential testing harness. Because all serialization regardless of the format ultimately produces a finalized string payload before leaving the Java method, the test harness can treat the resulting HTTP response body as an opaque textual blob. It simply verifies the candidate's output byte-for-byte against the parent commit's baseline, ensuring absolute formatting consistency without needing format-specific parsing logic in the test suite.

3.1.2 OpenAPI and Swagger UI

The contract of the production API is defined in an OpenAPI Specification (OpenAPI) 3.1.0 [19] document (`openapi.json`) packaged within the WAR. This document powers a Swagger User Interface (UI) [20] instance hosted at `/swagger.html`, providing an interactive, in-browser environment to explore and test the endpoints.

This setup yields two primary benefits. First, the document serves as the definitive source of truth for downstream consumers of the API, most notably the EPOS portal. Second, it enables future automated contract testing: a planned evolution of the test framework, discussed in Chapter 6, involves dynamically deriving HTTP differential test cases directly from this OpenAPI specification, rather than maintaining them as separate, static JSON definitions.

3.2 Java and Jakarta EE

The applications developed in this project target Java SE 21 [21]. The decision to utilize Java was effectively dictated by the existing API codebase. However, modern language features are leveraged extensively: `record` types are used liberally to model immutable request and response Plain Old Java Objects (POJOs), while the new pattern-matching `switch` expressions replace several complex `if/else` chains inherited from the legacy system, resulting in more maintainable code.

Jakarta EE 11 [4] serves as the foundational platform layer, with the codebase fully migrated to the modern `jakarta.*` namespace. The production application actively relies on a focused subset of core specifications: Jakarta REST 4.0 (Jakarta RESTful Web Services (JAX-RS)) [22] is used strictly for routing endpoints, while Jakarta *Contexts and Dependency Injection* (CDI) 4.1 handles all dependency injection (deliberately eschewing the standard JAX-RS `@Context` injection). Furthermore, the application utilizes Jakarta

Persistence 3.2 (Jakarta Persistence API (JPA)) [23] for object-relational mapping and Jakarta Bean Validation 3.1 for declarative constraints. Notably, standard Jakarta JSON Binding is omitted in favour of explicit serialization via Jackson, as detailed in Section 3.1.1.

During the early stages of the project, development prototypes served as a learning sandbox to evaluate the broader capabilities of the Jakarta ecosystem. These initial iterations experimented with Eclipse MicroProfile [24] features, specifically JSON Web Token (JWT) 2.1 for authentication mechanics and Health 4.0 for application probes. While these tools provided valuable insight into the platform's offerings, they were ultimately excluded from the final application in favour of a leaner, more tailored architecture.

Finally, alternative frameworks such as Spring Boot [25] were briefly considered but rejected. The existing production codebase is already built upon Jakarta EE and deployed on an established Payara server infrastructure. Attempting a parallel migration to a completely different ecosystem would have introduced significant risk and overhead without providing any proportional technical benefit.

3.3 SQL and the Database Layer

The persistence model of the production API is heavily relational: GNSS stations, networks, agencies, countries, file types, observation summaries, and embargo windows are all inextricably linked through foreign keys. Because users require highly specific data filtering, most endpoints must execute complex, conditional joins per request.

While JPA is utilized for its foundational object-relational mapping, the sheer complexity of the API's query requirements makes purely object-oriented querying (such as the JPA Criteria API) unwieldy. Instead, developers actively construct dynamic, hand-made queries through conditional string concatenations directly in Java. These hand-crafted strings are then passed to the JPA `EntityManager`, which safely binds the parameters and maps the resulting data rows back into Java POJOs. Consequently, native Structured Query Language (SQL) and its structural logic remain the true driving force of the database layer.

The laboratory has standardized on **PostgreSQL** [26] for the EPOS production database. The decision to use PostgreSQL over alternatives like **MySQL** [27] was driven primarily by historical precedent within the lab, as well as its robust support for specialized index types (such as B-tree, BRIN, and GiST) that are highly effective for large scientific datasets. Furthermore, while the PostGIS extension is not actively utilized today, standardizing on

PostgreSQL provides a realistic, low-friction pathway for future spatial data integrations.

3.4 Servers, Tooling and Network

Payara Server [5] is the Jakarta EE application server adopted by the laboratory; the production API is simply deployed onto an existing Payara installation. Its companion product, **Payara Micro**, is a single, self-contained Java Archive (JAR) that launches a deployable WAR directly from the command line. Because Payara was already the established stack, adopting Payara Micro for test orchestration was the immediate and obvious choice. It serves as the engine for the HTTP-level differential harness, booting one instance per WAR on different ports to safely replay and compare requests.

The applications are built using **Apache Maven**. The project's `pom.xml` orchestrates the build lifecycle, utilizing the `maven-war-plugin` to produce the deployable `ROOT.war`, the `maven-failsafe-plugin` for integration testing (outputting JUnit XML reports), and the `maven-dependency-plugin` to stage the Payara Micro JAR so the differential harness can spawn it as a subprocess.

GitLab [16] handles both source control and CI/CD orchestration. The laboratory hosts a self-managed GitLab instance [18], while the runner operates on a separate internal Virtual Machine (VM) executing jobs inside **Docker** [28] containers. Pipeline jobs utilize the official `maven:3.9.6-eclipse-temurin-21` image to guarantee a clean, reproducible toolchain. Under the hood, **Git** [13] is used heavily in the pipeline: the `git worktree` command materializes the baseline build alongside the candidate build on the runner's filesystem without disrupting the working copy (detailed in Section 5.2).

During early development, local database instances were simulated using Docker containers. However, this was later abandoned in favor of connecting directly to the real development database. Because the GitLab runners are provisioned on the same internal network as the EPOS database, connecting to the actual infrastructure was the most obvious and reliable choice for integration testing. The production API itself remains uncontainerized, deployed traditionally by copying the WAR onto the existing Payara server.

Finally, the production database is strictly isolated within the laboratory's internal network. While the internal runners can access it directly, developers operating remotely must connect via the university's established **OpenVPN** infrastructure, managed through pfSense. This setup utilizes existing UBI cer-

tificates, relying on proven, pre-configured network infrastructure rather than introducing new VPN solutions to the laboratory.

3.5 Conclusion

The technology stack on which this project is built is in large part imposed by its environment: Jakarta EE on Payara, PostgreSQL on the internal network, GitLab as the self-managed CI platform. Where there was room for choice, between Jakarta EE and Spring Boot, between Payara Server and Apache Tomcat, between PostgreSQL and MySQL, between JUnit 5 and TestNG, the decisions documented in this chapter were guided by a preference for fewer moving parts, by the alignment with what the laboratory already runs in production and by the desire to keep the differential testing harness as simple as possible. The next chapter describes the iterative methodology by which these technologies were combined into a working solution.

Chapter

4

Methodology and Architecture

4.1 The Legacy API

The starting point of the work is a code base that had grown organically for several years and that nobody fully understood anymore: the previous version of the EPOS API contained Java methods enormous scale, and source files of an even larger scale, and many subtle behavioral quirks introduced as patches by successive developers in response to specific bug reports. Consequently, the laboratory could not confidently guarantee that an arbitrary change would avoid silently breaking downstream API clients. While the full rewrite to Jakarta EE 11 significantly improved the internal architecture introducing concise methods, cohesive JAX-RS resources, and a strict separation between parsing, validation, querying, and serialization it also introduced a major regression risk. At any given commit during the refactoring process, an endpoint might produce output subtly different from the production baseline. The objective of this project is to construct a testing harness that mitigates this risk, turning subtle data deviations into immediate, actionable failures within the CI pipeline.

4.2 Iterative Methodology

The work was organised into three iterations, each carried out in its own dedicated repository. This separation served two purposes. First, it kept each iteration small and focused: the first prototype is deliberately stripped down to two trivial endpoints, so that the entire complexity is in the CI configuration and not in the application; the second prototype is the opposite, focusing on application code and leaving CI aside; the production API then brings the two

strands back together. Second, the prototypes have value in their own right as didactic artifacts that future students of the laboratory can read and re-use.

4.3 Prototype Applications

The first prototype, `java-ee-app`, validated the CI/CD infrastructure end-to-end before any of the production code base was touched: it is a minimal Maven WAR exposing two trivial JAX-RS endpoints, with a `.gitlab-ci.yml` that builds the artifact and runs Arquillian integration tests against two managed Payara Server containers declared as *services* in the job. Three patterns from that pipeline were carried over verbatim to the production one: the pinned base image with the `ubi-runner` tag, the global Maven repository cache, and the `reports: junit:` stanza that makes GitLab understand assertion failures as failed pipeline steps.

The second prototype, `syncspace-jakarta-app`, focused on the opposite concern: a complete Jakarta EE 11 application with no CI pipeline, exercising the specifications listed in Section 3.2 on a small room-booking domain (entities `User`, `Room`, `Booking`) protected by MicroProfile JWT [24]. JAX-RS [22] declares the resources, CDI wires the services, JPA [23] maps the entities, Bean Validation enforces a custom `@ValidDateRange` constraint, and a focused set of four `ExceptionMappers` (`BusinessValidationMapper`, `ValidationExceptionMapper`, `JsonProcessingExceptionMapper`, `GlobalExceptionMapper`) translates domain and framework exceptions into structured JSON errors without leaking stack traces. The end-to-end exception flow is summarised in Figure 4.1.

Two patterns from `syncspace-jakarta-app` were adopted by the production refactor: the use of a small set of focused `ExceptionMappers` (which keeps HTTP status codes meaningful and gives the differential harness a stable signal to assert on) and the use of `record`-based POJOs for request and response bodies (which simplifies diffing). The remaining features, in particular the full JWT flow, were retained in the prototype as a self-contained reference example for future students.

4.4 Architecture of the Differential Testing Framework

The architecture of the final testing framework, applied to the production API, is summarised in Figure 4.2. It is made of three concentric layers: the GitLab pipeline, the harness processes, and the application instances themselves.

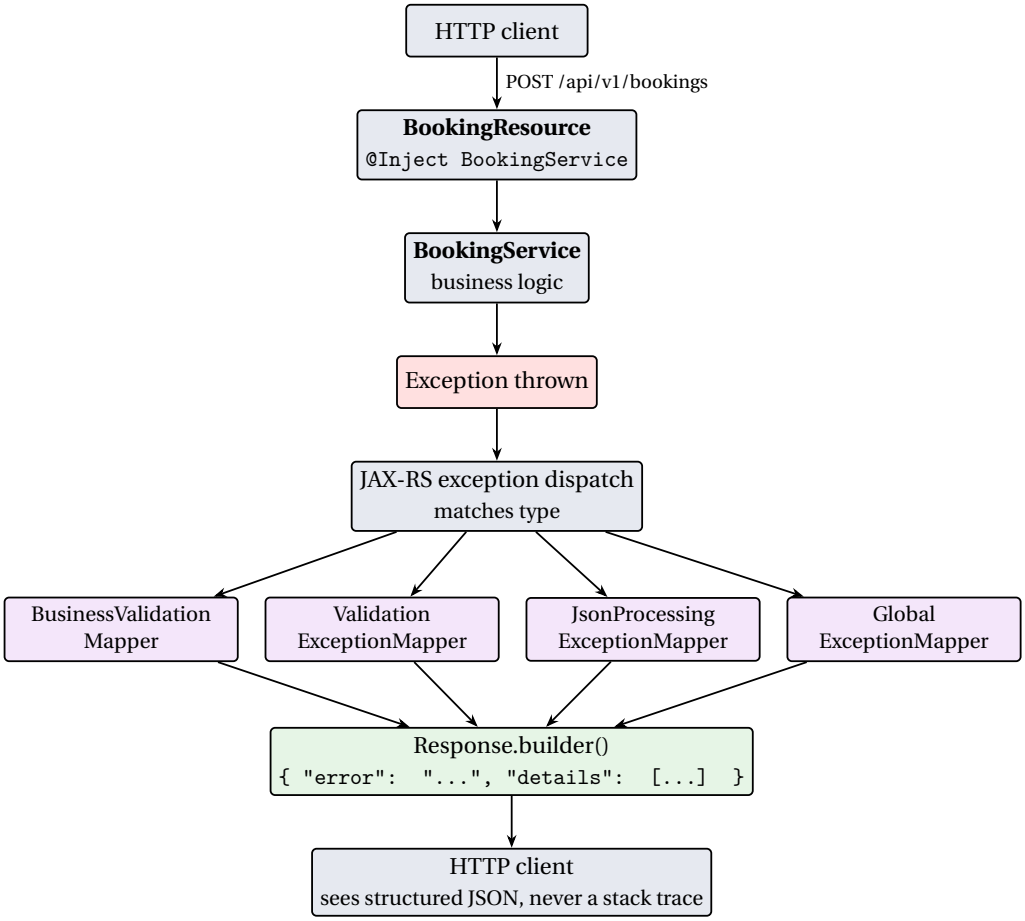

Figure 4.1: The CDI + JAX-RS exception-handling flow JSON error body.

Solid arrows denote control flow inside the pipeline; dashed arrows denote artifact or data access. The `http-diff-job` runs a three-way comparison between the `HEAD` build and two baselines, the immediate parent commit and a fixed reference commit, all pointed at the same database fixture.

A push by the developer triggers a pipeline on the runner tagged `ubi-runner-.31`, which goes through three jobs across three stages: `build-job` produces the `HEAD` WAR, `integration-test-job` runs the regular JUnit integration tests, and `http-diff-job` reconstructs both baseline WARs via Git worktrees, boots all three Payara Micro instances on different ports, replays `http-diff-cases.json` against each and emits per-baseline diff reports. All reports are picked up by GitLab as both JUnit results and raw artifacts.

The two prototypes traded a longer time-to-result for a sharper separation

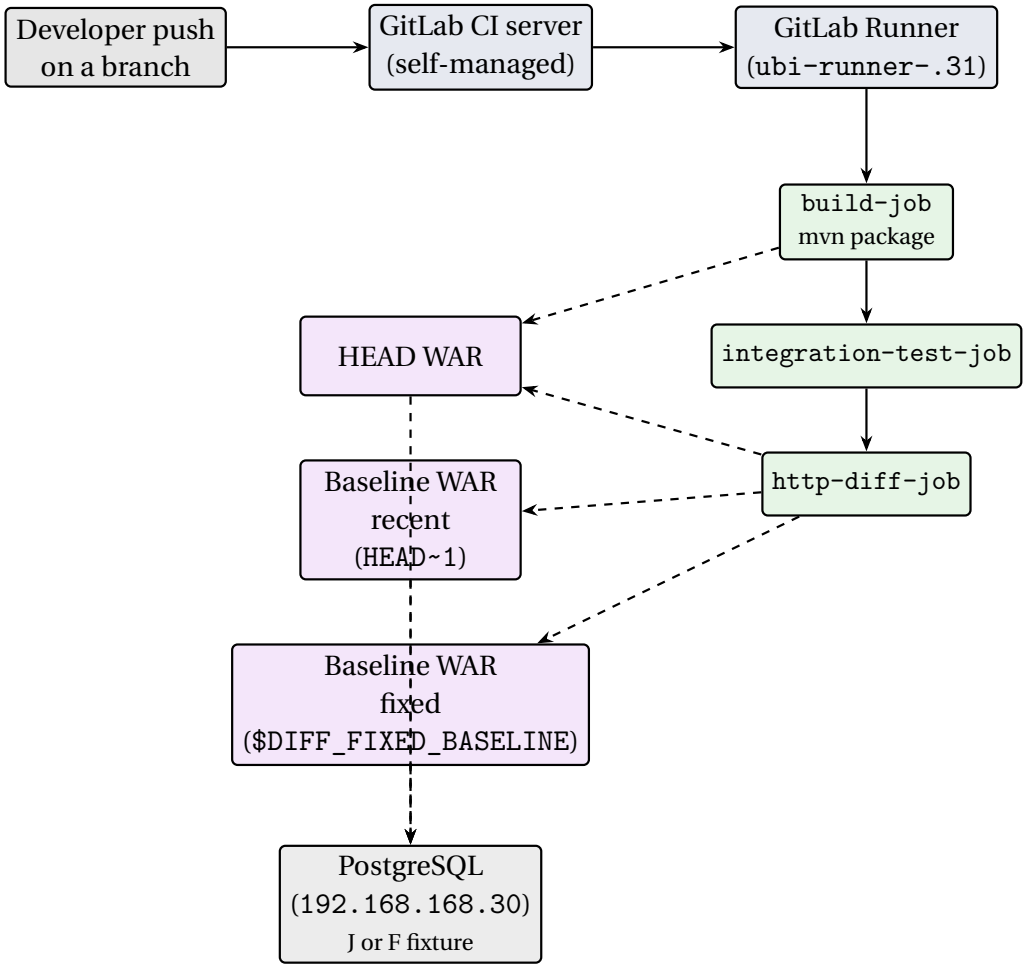

Figure 4.2: Overall architecture of the differential testing framework.

of concerns between CI/CD infrastructure and Jakarta EE feature usage; the next chapter applies the resulting framework to the production API.

Chapter

5

Use Case: EPOS API

This chapter applies the framework of Chapter 4 to the production API, `api_v4.0`. The EPOS itself [1] is a pan-European research infrastructure in solid Earth science, organised into Thematic Core Services for seismology, geomagnetic observations, GNSS data and products [2], and so on; the SEGAL laboratory of UBI contributes to the GNSS service by operating a node that hosts station metadata and RINEX [3] observation files, and `api_v4.0` is the public interface of that node. Three properties of the domain shape the test framework: the entities are relatively static (stable baselines are easy to obtain), the typical client request returns a long list of records (the comparison algorithm must scale to long diffs), and the same information is exposed in several serialization formats (the surface area to test multiplies but the underlying logic does not).

5.1 Architecture of `api_v4.0`

The application is a Maven WAR project targeting Java SE 21 and Jakarta EE 11, deployed onto Payara Server 6 with `finalName=ROOT` so that it serves the root context. Its source tree is organised into four top-level packages that form a classical layered architecture (Figure 5.1): `Glass` provides the REST layer (JAX-RS resources `StationEndpoints`, `FilesEndpoints`, `ListEndpoints` and `Start`, plus parsers, format writers and the validation/shape helpers); `Geometry` and the helper classes `Validate.java` and `Shape.java` form a thin business layer; `DB_Tables` holds the forty JPA entities that model the relational schema (stations, networks, agencies, file types, observation summaries, embargoes); and `Config` (`Loader.java`) is the cross-cutting concern that handles bootstrap.

Solid arrows denote ordinary call/data flow; dashed arrows denote configuration and infrastructure binding established at deployment time.

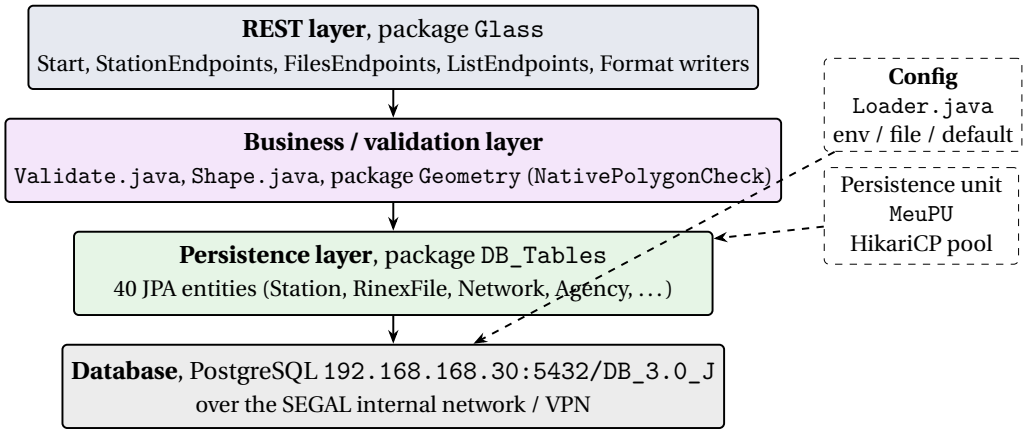

Figure 5.1: Layered architecture of EPOS V4.

5.1.1 Endpoints and Output Formats

The REST surface is declared in four classes inside the Glass package. Start.java carries the @ApplicationPath("/api") annotation and registers a PingResource for the trivial /api/ping liveness check. The three substantive resources are StationEndpoints (/api/stations, seven endpoints for station metadata indexed by identifier, location, agency, network, equipment or coordinates), FilesEndpoints (/api/files, four endpoints for RINEX file metadata filtered by agency/antenna/bounding box/date, by marker, the high-rate variant, and an aggregated combination), and ListEndpoints (/api/list, a parametric endpoint returning the valid values of a given filter). Every endpoint accepts a {format} path parameter and emits JSON, XML, CSV or (for the file endpoints) a downloadable shell script; the same OpenAPI 3.1.0 document is served at /swagger.html through a Swagger UI hosted inside the WAR.

5.1.2 Persistence

The persistence unit, declared in src/main/resources/META-INF/persistence.xml, is named MeuPU (inherited from the legacy code). It is configured as RESOURCE_LOCAL, with Hibernate as the JPA provider and a connection pool managed by HikariCP. Database connection parameters, host, port, database name, user, password, are resolved at

startup by `Config/Loader.java`, with the following precedence: environment variables, then a `Config.config` file at the root of the deployment, then hard-coded defaults pointing at the laboratory's database (`192.168.168.30:5432`).

The laboratory maintains two database fixtures with the same schema but different volumes of data:

- `DB_3.0_J` hosts a small but representative production subset, on the order of ten stations, eighty-six networks, two hundred and thirty agencies and roughly one hundred and forty-six thousand `rinex_file` rows at the time of writing. This is the fixture used by the correctness diff runs, because it is small enough to allow per-case comparison of full response bodies without overwhelming the report.
- `DB_3.0_F` hosts the full dataset and is used by the volume-oriented diff runs, where the goal is not byte-for-byte equality but the detection of changes that would only be apparent at scale, for example, an inadvertent `N+1` query introduced by a refactor, or a query plan regression visible only on a large table.

The precedence rules of `Loader.java` are what allow the `http-diff-job` to switch the running WARs between the two fixtures without rebuilding the artifact: the job sets the appropriate environment variables in its `variables:` block and the application picks them up at boot.

The shape of the schema is discussed in Section **??**; a simplified view of the central entities is given in Figure 5.2.

Representative columns as they appear in the live PostgreSQL schema. Green entities carry domain data; purple entities are controlled-vocabulary lookups (note that `country.iso_code` is the natural primary key, not a surrogate `id`); grey entities are junction tables that materialise the many-to-many relationships of the model. Arrows point from the "one" side to the "many" side of the relationship.

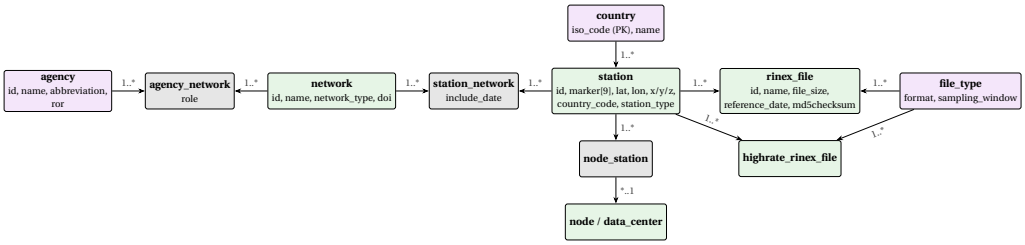

Figure 5.2: Simplified view of the core EPOS data model

5.2 The CI/CD Pipeline

The pipeline of `api_v4.0` extends the patterns validated on the first prototype with an additional job in a new `diff` stage. The diff job compares the current build against *two* baselines, in what amounts to a three-way comparison: the immediate parent commit `HEAD~1` (which catches regressions introduced by the commit currently being pushed) and a *fixed* reference commit identified by a SHA stored in the `DIFF_FIXED_BASELINE` GitLab CI/CD variable (which catches the slow drift that accumulates over many commits along a long-running refactor branch). The key parts of the `.gitlab-ci.yml` are shown, condensed, in Listing 5.1.

```
default:
  image: maven:3.9.6-eclipse-temurin-21
  tags: [ ubi-runner-.31 ]
cache: { key: global-maven-cache, paths: [ .m2/repository ]
   }
variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/
     repository"
  GIT_DEPTH: "20"
stages: [ build, test, diff ]

build-job:
  stage: build
  script: [ mvn clean package -DskipTests ]
  artifacts: { paths: [ target/*.war ] }

integration-test-job:
  stage: test
  script: [ mvn verify ]
  artifacts:
    when: always
    reports: { junit: [ "target/failsafe-reports/TEST-*.xml"
        ] }

http-diff-job:
  stage: diff
  needs: [ { job: build-job, artifacts: true } ]
  variables:
    DATABASE_URL: "postgres://postgres:postgres@192
       .168.168.30:5432/DB_3.0_J"
    JAKARTA_SCHEMA_ACTION: "none"
    DIFF_FIXED_BASELINE: ""
  script:
    - |
      # build baselines if available, then let HttpDiff
         orchestrate
```

```
    # spawning Payara Micros on ports 8080/8081/8082.
    [ -n "$(git rev-parse --verify HEAD~1 2>/dev/null)" ]
       && \
        export BASELINE_RECENT_WAR=$(build recent HEAD~1)
    [ -n "$DIFF_FIXED_BASELINE" ] && \
        export BASELINE_FIXED_WAR=$(build fixed "
           $DIFF_FIXED_BASELINE")
    mvn test -Dtest=HttpDiff -DfailIfNoTests=false
artifacts:
  when: always
  paths:
    - target/http-diff-report/
  reports:
    junit:
      - "target/surefire-reports/TEST-Diff.HttpDiff.xml"
  expire_in: 4 weeks
```

Code Listing 5.1: Condensed view of the production `.gitlab-ci.yml`, three stages and three jobs.

Three details of the listing are worth highlighting. The *recent* baseline is `HEAD~1`, which keeps the per-commit comparison focused on the change just introduced; the *fixed* baseline is a commit Identifier (ID) stored in `DIFF_FIXED_BASELINE` (typically pointing at the last commit known to have shipped to the EPOS portal) and acts as a long-term reference against which the cumulative drift of the branch is measured. Each baseline build is materialised through `git worktree` rather than `git checkout`, so the working copy is never disturbed and the harness can freely compare files from both checkouts in parallel. Finally, each baseline is optional and treated independently: if `HEAD~1` does not exist (first commit of the repository) or `DIFF_FIXED_BASELINE` is unset (start of a refactor branch), the corresponding sub-run is skipped. A pipeline can therefore have zero, one or two diff sub-runs depending on the situation, and never fails because a baseline is simply unavailable.

5.3 The HTTP-Level Diff: `HttpDiff.java`

`HttpDiff.java` is the harness that exercises the API end-to-end. Its execution proceeds in four phases, summarised graphically in Figure 5.3. The HTTP harness runs once and internally boots *all* the available WARs in parallel (the `HEAD` WAR, the recent baseline if any, and the fixed baseline if any). It then issues every test case against every running instance and emits one diff report per baseline.

The harness boots one Payara Micro per available WAR, the `HEAD` build and whichever of the recent/fixed baselines were provided, waits for all of them to be ready, fires every test case against every instance in parallel, then performs a three-way comparison and emits one diff report per baseline.

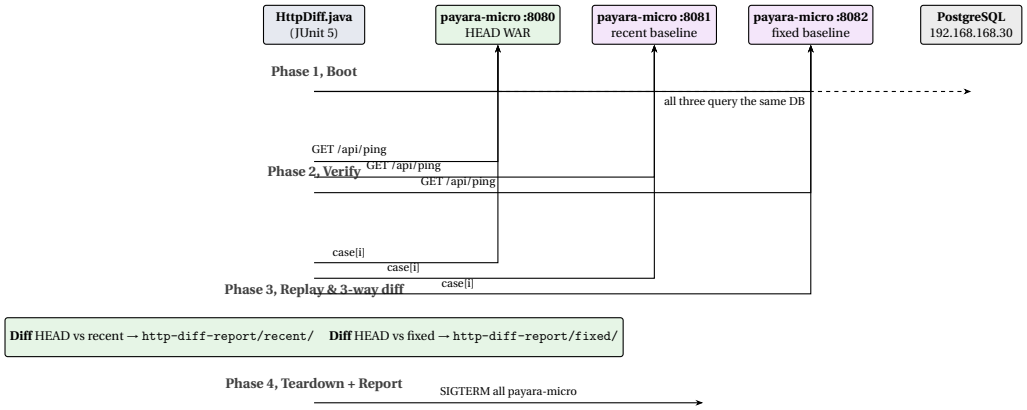

Figure 5.3: Phases of `HttpDiff.java`.

The figure already covers the gist; a few implementation details are worth recording. The test cases live in `src/test/resources/http-diff-cases.json` as a JSON array of request descriptors (method, path, headers, optional body); This guards against spurious failures from requests landing on still-deploying containers. Bodies are compared as normalised strings (sorted JSON keys, trimmed whitespace) and the headers that vary by definition (`Date`, `Server`, `Content-Length`) are ignored. The two comparisons (HEAD vs recent, HEAD vs fixed) are kept in separate report directories so that a per-commit regression and an accumulated drift remain distinguishable in the artifact tree.

5.4 Reports and Findings

Each pipeline run publishes the integration-test reports, the diff reports (`recent/` and `fixed/` subdirectories of `target/http-diff-report/`) and the JUnit XML files consumed by GitLab. Integration test results expire after one week; the diff reports are kept for four because they are the historical record of behavioural change across the refactor. During development the harness surfaced real regressions hidden inside seemingly cosmetic refactors that no line-by-line review of the diff would plausibly have caught by eye. The next chapter draws the main conclusions of the project and outlines the items that, by choice or by constraint, were left out of its scope.

Chapter

6

Conclusions and Future Work

6.1 Main Conclusions

The work described in this report addresses a concrete problem of the SEGAL laboratory: how to refactor a large, several-years-old Jakarta EE API without silently breaking the EPOS clients that depend on it. The answer is a three-stage CI/CD-driven testing framework that, for every commit, compares the new build against two baselines (the immediate parent commit and a fixed long-term reference) at the level of HTTP responses.

A few conclusions stand out. *Differential testing fits a refactor naturally*: asserting that the new build produces the same output as a known-good one sidesteps the brittle fixture problem and made the framework useful from the very first commit it ran on. *Comparing whole HTTP responses catches what unit tests miss*: the HTTP-level harness surfaces integration regressions that appear only when the full stack runs against a live database, the class of change a refactor is most likely to introduce and that isolated unit tests are least likely to reveal. *A self-hosted pipeline is not optional here*: only a runner inside the laboratory's network can exercise the API end-to-end against the production database, so self-managing both the GitLab control plane and the GitLab Runner host followed from the data-locality constraint, not from a preference. *The two prototypes paid for themselves*: each answered a small, well-bounded set of questions in an environment where mistakes were cheap, sparing the production code base from them. *The framework reveals what a human review would miss*: during the refactor it surfaced behavioural changes buried inside seemingly cosmetic commits that no amount of code review would plausibly have caught at the laboratory's commit rate.

6.2 Future Work

Several extensions to the framework were deliberately left out of scope. The most immediate are the introduction of **BRIN** indexes on the date columns of `rinex_file` (and the companion *performance-regression subsystem* that would record wall-clock durations against the `F` fixture to justify them empirically), the **PostGIS migration** of the spatial helpers in the `Geometry` package (with the same differential framework used to verify byte-for-byte equivalence), and the **automatic generation of HTTP test cases** from the OpenAPI document shipped inside the WAR (so newly added endpoints are exercised by the diff from the moment they are merged). The framework would also benefit from a **schema diff** of the OpenAPI document itself across builds (to surface contract regressions distinct from behavioural ones), from a **snapshot test** layer for endpoints that should remain stable across releases (such as `/api/ping`), and from **containerization of the production deployment** to remove the residual gap between the WAR that was tested and the WAR that ships. Finally, the harness code itself could be **generalised into a small library** reusable by other Jakarta EE services of the laboratory, with the application-specific bits factored into a clean configuration interface; orthogonal to that, several **operational aspects** (transient-error retries, clean termination of orphan Payara Micro processes, automated provisioning of the runner host, a long-term dashboard on top of the diff reports) deserve more attention than the present project could afford.

Overall, the framework delivered by this project addresses the problem that motivated it, has already proved its value on a real regression caught during the refactor, and lays the groundwork for the laboratory to continue evolving the API with a much higher degree of confidence than was previously possible. The items listed in this section are natural next steps; none of them invalidates the design that was put in place.

Appendix

A

Raw Survey Data

Here is the table containing all 500 responses...

Bibliography

[1] EPOS ERIC. European Plate Observing System — EPOS, 2024. [Online] https://www.epos-eu.org/. Accessed June 2026.

[2] EPOS GNSS Data and Products Thematic Core Service. EPOS GNSS Data Gateway, 2024. [Online] https://gnss-epos.eu/. Accessed June 2026.

[3] Werner Gurtner and Lou Estey. RINEX — The Receiver Independent Exchange Format, Version 3.05, 2020. International GNSS Service (IGS), RINEX Working Group.

[4] Eclipse Foundation. Jakarta EE 11 Platform Specification, 2024. [Online] https://jakarta.ee/specifications/platform/11/. Accessed June 2026.

[5] Payara Services Ltd. Payara Platform Community Documentation, 2024. [Online] https://docs.payara.fish/. Accessed June 2026.

[6] SEGAL Laboratory. GNSs V4 Production API Repository. https://gitlab.com/segalubi/api_v4.0, 2026. Access restricted to laboratory members.

[7] Eduardo Abrantes. Java EE Prototype Application Repository. https://gitlab.com/PatalJunior/java-ee-app, 2026.

[8] Eduardo Abrantes. Syncspace Jakarta Prototype Application Repository. https://gitlab.com/PatalJunior/syncspace-jakarta-app, 2026.

[9] JUnit Team. JUnit 5 User Guide, 2024. [Online] https://junit.org/junit5/docs/current/user-guide/. Accessed June 2026.

[10] Cédric Beust. TestNG documentation, 2024. [Online] https://testng.org/. Accessed June 2026.

[11] Red Hat, Inc. Arquillian Container Testing Framework, 2024. [Online] https://arquillian.org/. Accessed June 2026.

[12] Twitter Engineering. Diffy — Automatically catch bugs in service-oriented architectures, 2015. [Online] https://blog.twitter.com/engineering/en_us/a/2015/diffy-testing-services-without-writing-tests. Accessed June 2026.

[13] Git Development Community. Git: Fast Version Control System. https://git-scm.com/, 2005. Original author: Linus Torvalds.

[14] Jenkins Project. Jenkins — The leading open source automation server, 2024. [Online] https://www.jenkins.io/. Accessed June 2026.

[15] GitHub Inc. GitHub Actions Documentation, 2024. [Online] https://docs.github.com/actions. Accessed June 2026.

[16] GitLab Inc. GitLab CI/CD documentation, 2024. [Online] https://docs.gitlab.com/ci/. Accessed June 2026.

[17] GitLab Inc. GitLab Runner documentation, 2024. [Online] https://docs.gitlab.com/runner/. Accessed June 2026.

[18] GitLab Inc. GitLab Self-Managed — Install and configure, 2024. [Online] https://about.gitlab.com/install/. Accessed June 2026.

[19] OpenAPI Initiative. OpenAPI Specification, Version 3.1.0, 2021. [Online] https://spec.openapis.org/oas/v3.1.0. Accessed June 2026.

[20] SmartBear Software. Swagger UI, 2024. [Online] https://swagger.io/tools/swagger-ui/. Accessed June 2026.

[21] Oracle Corporation. *Java Platform, Standard Edition 21 API Specification*. Oracle Corporation, 2023. [Online] https://docs.oracle.com/en/java/javase/21/.

[22] Eclipse Foundation. Jakarta RESTful Web Services 4.0, 2024. [Online] https://jakarta.ee/specifications/restful-ws/4.0/. Accessed June 2026.

[23] Eclipse Foundation. Jakarta Persistence 3.2, 2024. [Online] https://jakarta.ee/specifications/persistence/3.2/. Accessed June 2026.

[24] Eclipse Foundation. Eclipse MicroProfile, 2024. [Online] https://microprofile.io/. Accessed June 2026.

[25] VMware, Inc. Spring Boot Reference Documentation, 2024. [Online] https://docs.spring.io/spring-boot/index.html. Accessed June 2026.

[26] PostgreSQL Global Development Group. PostgreSQL 16 Documentation, 2024. [Online] https://www.postgresql.org/docs/16/. Accessed June 2026.

[27] Oracle Corporation. MySQL 8.0 Reference Manual, 2024. [Online] https://dev.mysql.com/doc/refman/8.0/en/. Accessed June 2026.

[28] Docker, Inc. Docker Documentation, 2024. [Online] https://docs.docker.com/. Accessed June 2026.